

qlib Integration — AI Analyst

Return & risk prediction, portfolio optimization, and score enhancement, powered by Microsoft qlib.

Document date: 2026-07-28 · Applies to *Desktop/AI_Analyst* (backend *api/quant/**, *committee*, and the *Quant* frontend tab).

1. Summary

The AI Analyst app previously had **no learned return model** — every expected return (μ) was a naive historical mean — and its live "interest score" was a single hand-weighted linear blend. A complete portfolio optimizer and risk stack existed but were **dormant** (unregistered). A full clone of qlib sat in the repo, imported nowhere.

This integration wires qlib in as a **lean, in-process library** (no `qlib.init`, no `.bin` data provider, no mlflow tracking) to deliver:

- Return prediction** — a cross-sectional LightGBM alpha model over the Postgres fundamentals panel.
- Risk prediction** — a factor-structured forward covariance (`StructuredCovEstimator`).
- Portfolio optimization** — both the existing **native** optimizer *and* qlib's optimizers, selectable at runtime.
- Score enhancement** — the learned alpha plus regime-conditioned factor-IC weights blended into the scanner.
- Committee integration** — a `quant_signals` evidence node and a `quant_factor_analyst` specialist.
- REST + UI** — a `/api/quant/*` router and a new **Quant** tab.

Everything degrades gracefully: with no trained model or qlib absent, the app runs exactly as before.

2. How qlib maps onto the needs

Need	qlib capability	Where it lives
Return prediction	<code>LGBModel + DatasetH + DataHandlerLP.from_df</code>	<code>api/quant/qlib_alpha.py</code>
Risk prediction	<code>StructuredCovEstimator($\Sigma = F \cdot cov_b \cdot FT + diag(var_u)$)</code>	<code>api/quant/qlib_risk.py</code>
Portfolio optimization	<code>PortfolioOptimizer, EnhancedIndexingOptimizer</code> (+ native <code>optimize.py</code>)	<code>api/quant/qlib_optimize.py</code>
Evaluation / backtest	<code>calc_ic, risk_analysis</code>	<code>qlib_alpha.evaluate, api/quant/qlib_backtest.py</code>

Lean-embed principle: qlib's model/risk/optimizer/eval run on a plain MultiIndex DataFrame we build from Postgres — we never touch qlib's expression engine, `qlib.data.D`, Alpha158/360, or the recorder/`qrun` workflow. Only `lightgbm`, `scikit-learn`, and `cvxpy` become hard new runtime dependencies.

3. New modules

File	Responsibility
<code>api/quant/qlib_data.py</code>	Build the monthly cross-sectional feature/label panel from Postgres (reuses the <code>cycle/ic.py</code> loaders + point-in-time alignment).
<code>api/quant/qlib_alpha.py</code>	AlphaLGB model (recorder-free), <code>train/predict/evaluate</code> , dill persistence, lazy cache, CLI.
<code>api/quant/qlib_risk.py</code>	<code>structured_cov()</code> → annualized $\Sigma + (F, cov_b, var_u)$ decomposition; sync price/returns loader.

<code>api/quant/qlib_optimize.py</code>	<code>solve(optimizer, ...)</code> dispatcher over native + 5 qlib backends → uniform <code>PortfolioSolution</code> .
<code>api/quant/qlib_backtest.py</code>	Cross-sectional signal backtest scored with qlib <code>risk_analysis</code> .
<code>api/quant/alpha_signal.py</code>	Central bridge: cached latest cross-section + per-ticker expected returns (used by scanner, committee, router).
<code>api/quant/ic_weights.py</code>	Regime-conditioned factor-IC → per-family scoring weights.
<code>api/routers/quant.py</code>	<code>/api/quant/{backends,alpha,risk,optimize,backtest}</code> .
<code>frontend/src/views/QuantView.tsx</code>	The Quant tab (optimizer-backend selector, alpha table, backtest).

Modified: `api/quant/risk.py` (qlib_structured covariance option), `api/routers/screener_agent.py` (alpha + IC-weighted scoring), `committee/{state,nodes,graph,archetypes}.py`, `api/main.py`.

4. Return prediction — the alpha model

Panel. `qlib_data.build_panel()` produces a monthly (`datetime, instrument`) panel: features are importance-ranked fundamentals (value / quality / growth / market-factor families) from `fact_metrics_*`, cross-sectionally z-scored per month; the label is the forward 1-month (or 3-month) stock return. Fundamentals are aligned point-in-time (`period_end + 90 days → month-end`) to avoid look-ahead — the same alignment the regime-IC engine uses, so the two share one data path.

Model. `AlphaLGB` subclasses qlib's `LGBModel` and overrides `fit` to drop qlib's recorder logging, so training needs **no qlib.init** and **no mlflow**. Training uses the real qlib `DatasetH` pipeline; prediction runs the stored LightGBM booster directly on any panel.

Train / use:

```
python -m api.quant.qlib_alpha train --jurisdiction US --start 2022-01-01
python -m api.quant.qlib_alpha predict --jurisdiction US --top 20
```

Artifacts (model + metadata: IC, trained_at, horizon, features) are dilled to `output/quant_models/` and lazily cached. Out-of-sample quality on US data: **rank-IC** ≈ **0.09–0.14** on forward 1-month returns.

5. Risk prediction — structured covariance

`qlib_risk.structured_cov(tickers, R)` runs `StructuredCovEstimator(factor_model="pca", scale_return=False)` on a daily return panel and returns an **annualized StructuredRisk**: the full covariance Σ , the factor decomposition (`F, cov_b, var_u`), and forward volatility $\sqrt{\text{diag}(\Sigma)}$. `scale_return=False` is essential — qlib's default rescales returns to percent and inflates variance 100²x.

It is exposed two ways: as a selectable `qlib_structured` model inside the existing `risk.build_risk_bundle`, and directly to the optimizer for the enhanced-indexing decomposition.

6. Portfolio optimization — dual backend (runtime choice)

`qlib_optimize.solve(optimizer, tickers, mu, sigma, ...)` dispatches to either backend and returns a uniform `PortfolioSolution` (weights, annualized expected return / vol / Sharpe, factor exposures, diagnostics):

optimizer	Engine	Notes
native	<code>api/quant/optimize.py</code> (SLSQP / cvxpy-MIP)	Full constraints: sector caps, vol cap, cardinality, turnover.
<code>qlib_mvo</code> / <code>qlib_gmv</code> / <code>qlib_rp</code> / <code>qlib_inv</code>	qlib <code>PortfolioOptimizer</code>	Mean-variance / min-variance / risk-parity / inverse-vol; long-only, fully invested.
<code>qlib_enhanced_indexing</code>	qlib <code>EnhancedIndexingOptimizer</code> (cvxpy + ECOS)	Benchmark-relative tracking-error; consumes the <code>StructuredCovEstimator</code> decomposition.

Both backends are fed the same μ (annualized alpha or historical mean) and Σ (qlib-structured, Ledoit-Wolf, or sample). Neither is a default-only path — both ship enabled and are chosen per request / from the UI selector.

7. Score enhancement — alpha + regime-IC weights

`screener_agent._rank` (the value + sentiment "interest score") now blends:

1. **v_alpha** — the qlib model's expected return, cross-sectionally normalized, as the **dominant** term (default weight 0.40) when the model covers the shortlist.
2. **Regime-IC family weights** — the fundamental score budget (value / growth) is re-split by how predictive each family has been *in the current macro regime* (see §8), instead of hard-coded constants.

The legacy formula is preserved **bit-for-bit** as the fallback when no model or IC data is available (verified by test). Each row now reports **alpha**, **alpha_percentile**, and a **score_components** breakdown; the response carries a **scoring** block describing the weights and model actually used.

8. How the macro-regime IC model output is used in forecasting

This section answers the specific question: **where does the regime-conditioned IC feed forecasting?**

The pipeline that produces it (pre-existing):

1. The **macro cycle model** (HMM / VAE in `xbrl_sec/sec/cycle/`) labels each month with a market regime and stores it in `fact_cycle_state_monthly` (e.g. `mid_expansion`, with probabilities).
2. `cycle/ic.py::compute_regime_factor_ic()` computes, **conditioned on that regime**, the Spearman rank **information coefficient (IC)** of every fundamental `metric_id` against forward 1m / 3m stock returns, and writes it to `fact_equity_factor_ic_regime` (~316k rows). This is the "IC macro model output": a regime-aware measure of *which factors have actually predicted returns in conditions like today's*.

Where it is consumed in forecasting (this integration):

- **Scoring reweighting — the one live use.** `api/quant/ic_weights.family_weights(jurisdiction)` reads the **latest date's** rows for the current regime, aggregates the per-metric IC to family-level emphasis (value / growth / quality by mean |IC|, normalized), and caches it. `screener_agent._rank` uses these weights to re-split the composite's fundamental budget. Net effect: **the interest score leans harder on the factor family that the macro-regime IC says is currently most predictive**, and reverts to fixed weights when the IC table is unavailable.

Where it is *not* (yet) used — important for accuracy:

- The **qlib LGBM alpha model does not consume the regime IC or the regime label** as a feature. It learns from factor *values* (including momentum / volatility from the market-factor family), but it is **regime-agnostic**.
- The **committee** receives macro *regime context* from the macro node (regime quadrant / tilt) and the new `quant_signals` block, but not the factor-IC table directly.

Data flow (text diagram):

```
HMM/VAE cycle model --> fact_cycle_state_monthly (regime label)
                        |
                        v
cycle/ic.py::compute_regime_factor_ic (Spearman IC vs forward returns, per regime)
                        |
                        v
fact_equity_factor_ic_regime
                        |
api/quant/ic_weights.family_weights() (latest regime → value/growth/quality emphasis)
                        |
                        v
screener_agent._rank() ---> regime-tilted interest score
```

Recommended next step (not implemented): make the alpha model *regime-aware* — add the regime label / probabilities (and optionally the regime-IC family emphasis) as conditioning features in `build_panel`, so the learned forecast itself adapts to the macro regime rather than only the score weights. This is the natural way to push the IC macro model deeper into the forecasting path.

9. LangGraph committee integration

- **State:** `InvestmentCommitteeState.quant_signals` (`committee/state.py`) + config toggle `enable_quant_signals`.

- **Evidence node:** `qlib_signals_node` (`committee/nodes.py`) computes, in the **prepare** phase (once, shared across providers), the ticker's alpha expected return + universe percentile + model rank-IC, and a peer-group factor-risk read (forward vol, factor exposures, min-variance weight). It is registered in `_add_prepare_nodes` and added to `_EVIDENCE_NODES`, so it auto-fans-out from the engine and auto-fans-in to every analyst; it degrades to `available: False` on any error. The block is surfaced in `_agent_payload`, so all tribunal analysts can cite it.
- **Specialist:** `quant_factor_analyst` (`committee/archetypes.py`) — reconciles the statistical alpha/risk signals with the fundamental/DCF thesis and flags alpha-vs-valuation disagreements.

10. REST API and frontend

Router `api/routers/quant.py` (registered under `/api/quant`), all handlers offloading qlib work to worker threads:

Endpoint	Purpose
<code>GET /backends</code>	Optimizer backends, risk models, alpha-model metadata.
<code>POST /alpha</code>	Expected returns for a universe (or the latest top-N cross-section).
<code>POST /risk</code>	Forward covariance / vol / factor exposures.
<code>POST /optimize</code>	Portfolio optimization; <code>optimizer</code> selects native or any qlib backend.
<code>POST /backtest</code>	Signal backtest with qlib <code>risk_analysis</code> .

Frontend — a new **Quant** tab (`QuantView.tsx`) with an **optimizer-backend selector**, risk-model and μ -source selectors, a live expected-returns table, an optimized-weights table, and a backtest panel.

11. Ideas / LLM wiring — how it works, and the fix this iteration

The **Ideas** section is not wired to a single LLM in the way it first appears. Its three sub-features use LLMs differently:

- **Quick scan (value + sentiment):** one LLM call per name for MD&A tone, using the **user's selected provider** (single by design). Now enriched with the qlib alpha and regime-IC weights (\$7).
- **Prompt screen:** the prompt→filters **translation fans out across every keyed provider** — each surfaces a different candidate set (the per-name "found by" badges), and the union is ranked once.
- **Group ranking:** a **deterministic quantitative composite** (z-scored P/E, EV/EBITDA, P/B, FCF yield, growth, margins — `committee/group.py::deterministic_ranking`). No LLM is needed for the ranking; the LLM only writes the optional narrative/memo, on the selected provider.

Root cause of the "only one LLM (DeepSeek)" impression — fixed. Two user-facing strings hardcoded the name "DeepSeek" even when another provider (e.g. Moonshot/Kimi) was selected:

- `committee/group_prompts.py GROUP_MEMO_OFFLINE` — the offline group memo (the caption in the screenshot). Now provider-agnostic: *"no API key for the selected provider"*.
- `api/routers/screener_agent.py` — the no-key scanner warning. Now names the **actually-selected provider** via `llm_providers.get(provider).label` (e.g. *"No Moonshot (Kimi) key ..."*).

These made a multi-provider app read as single-provider. Behavior was already provider-correct; the copy now matches it.

Optional enhancement (not implemented): a full multi-provider group *verdict* fan-out — each keyed provider writing its own ranked narrative, like the Analyze/Compare committee debates. Today the group verdict runs on one provider because the ranking itself is deterministic and identical across providers.

12. Operational notes

- **qlib on Python 3.13** (officially unsupported; no wheel): build the bundled clone editable — `pip install -e ./qlib --no-build-isolation --no-deps` (needs the MSVC/VS2022 C++ toolchain). Lean runtime deps are pinned in `backend/requirements.txt` (`lightgbm`, `scikit-learn`, `cvxpy`, `ecos`, `mlflow`, `gym`, `pyarrow`, `dill`, `python-redis-lock`, ...).
- **Restart the backend** to pick up the new router; the frontend gains the Quant tab on rebuild.
- **Cold start:** the first alpha call for a jurisdiction builds the cross-section panel from the warehouse (~35–40 s), then caches it for 1 h (`QLIB_ALPHA_CACHE_TTL`). Consider pre-warming on startup for production.

- **Retraining:** the alpha model is retrained via the CLI; wiring the existing APScheduler dependency to a periodic retrain is a recommended production follow-up. One demo model (trained on 2022–2024) ships in `output/quant_models/`.
- **Tests:** `backend/api/quant/tests/test_quant.py` (12 tests: optimizer backends, structured covariance, exact scoring fallback, alpha blend, IC re-split, and a DB-gated panel/train smoke test).

13. Known limitations & recommended follow-ups

1. **Regime-aware alpha** — feed the macro regime / regime-IC into `build_panel` so the alpha *forecast* is regime-conditioned (§8), not only the score weights.
2. **Alpha cold-start** — pre-warm the cross-section cache on server startup.
3. **Scheduled retrain** — connect APScheduler to `qlib_alpha.train_and_save`.
4. **Multi-provider group verdicts** — optionally fan the Ideas group narrative across all keyed providers.
5. **JP coverage** — a JP alpha model is supported by the code but not yet trained.